
CS771 Minor Assignment 3: Melbo's 'Mazing Mixtures

Harpuneet Singh

Roll Number: 240426
harpuneets24@iitk.ac.in

Aarya Bhatt

Roll Number: 240016
aaryajb24@iitk.ac.in

Laksh Dhir

Roll Number: 240588
laksh24@iitk.ac.in

Aayush Gajeshwar

Roll Number: 240024
aayushk24@iitk.ac.in

Harshit Saini

Roll Number: 240442
harshits24@iitk.ac.in

Jashan Mittal

Roll Number: 240488
jashanm24@iitk.ac.in

Problem 2.1: Melbo's 'Mazing Mixtures

We used the mixed regression lecture code (alternating optimization from `lec16.zip`) and built a simple pipeline on top of it. First we filtered valid timestamps from `DD1(Oct).csv`, then converted the `Time` column into hour-of-day. After that we did z-score normalization, but only using training data stats, so no data leakage happened.

Before doing the main experiments, we tried different Ridge values $\alpha \in \{0.001, 0.01, 0.1, 1.0\}$ with `n_components = 4` and `n_iter = 90`. At convergence, $\alpha = 0.001$ and $\alpha = 0.01$ give nearly identical MAE (10.17 vs. 10.17), so we chose $\alpha = 0.01$ as it produces more stable Ridge fits across different numbers of components.

All five parts used same split: sequential 80/20 (first 80% timestamps for training, rest for testing). For evaluation, we did not use oracle assignment (which is unrealistic because it picks best model per point). Instead, we trained a `LinearSVC` on EM hard assignments and used it to decide which regression model to use for each test point. We set $C = 0.1$, which helped avoid overfitting but still gave enough capacity to separate clusters.

Part 1: How Many EM Iterations?

We fix `n_components = 4` (with the lecture's initialization bins `[0, 10, 25, 70, 200]`) and vary `n_iter` from 1 to 200 (every integer from 1 to 100, then in steps of 10 up to 200). This gives a dense view of the convergence curve while also confirming that no further changes happen beyond 100 iterations.

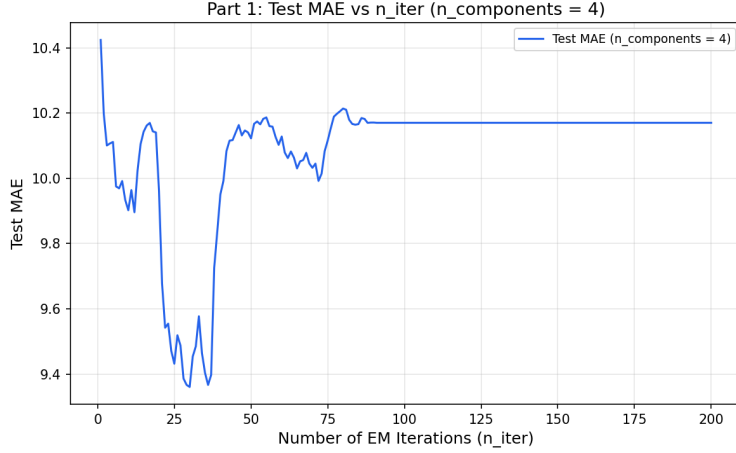


Figure 1: Test MAE as a function of n_iter with $n_components = 4$.

Range of large variation: Test MAE varies substantially in the range $n_iter \in [1, 80]$. It starts at 10.42 (at $n_iter=1$), drops to a transient minimum of 9.36 around $n_iter=30$, rises back up past 10.0, and continues fluctuating until roughly $n_iter=80$. These fluctuations are characteristic of hard-assignment EM: at each iteration, data points are reassigned to the component whose regression model fits them best, and a batch of such reassignments can temporarily improve or worsen the downstream LinearSVC classifier’s routing accuracy.

Knee point: Past $n_iter \approx 90$, the test MAE locks in at ≈ 10.17 and does not change even if we increase n_iter all the way to 200. The hard assignments have fully converged at this point and no data points swap clusters anymore, so additional iterations have zero effect on test performance. We therefore identify $n_iter = 90$ as the knee point and use this value in all subsequent parts.

Part 2: How Many Components?

We fix $n_iter = 90$ and sweep $n_components \in \{1, 2, 3, 4, 6, 8, 12, 16, 24, 32, 48, 64\}$, that is, the six required values plus six intermediate ones to map the transition more precisely. For values other than 4 we construct initialization bins from quantiles of the training O_3 distribution.

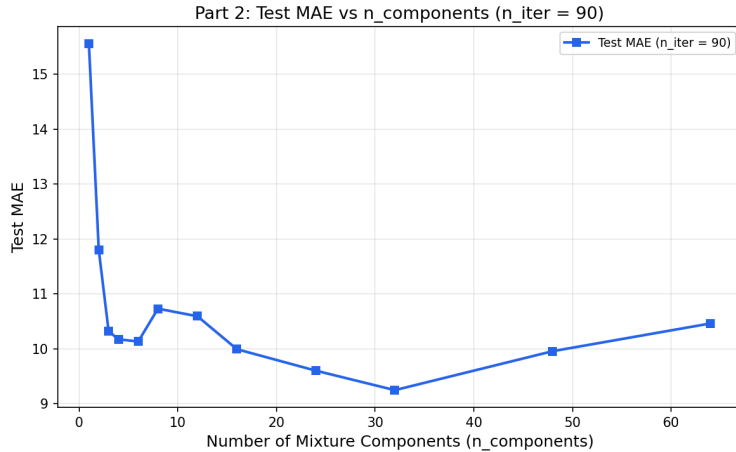


Figure 2: Test MAE vs. $n_components$ with $n_iter = 90$.

Observations. With $K = 1$ the model is a single Ridge regressor, and its MAE of 15.55 confirms that one linear model cannot handle the nonlinear sensor-to- O_3 mapping. Adding components brings

rapid improvement: MAE falls to 11.80 ($K = 2$), 10.32 ($K = 3$), and reaches **10.17** at $K = 4$. Performance stays in a similar range at $K = 6$ (10.13), then rises at $K = 8$ (10.73).

Interestingly, at higher K the MAE dips again, reaching its minimum of **9.25** at $K = 32$. With the fully converged EM assignments (at 90 iterations), additional components can capture finer subdivisions of the data that the classifier routes reasonably well at this scale. However, past $K = 32$ the performance degrades again ($K = 48$: 9.95, $K = 64$: 10.46). With too many components, some clusters end up too small for the Ridge model to fit well, and the LinearSVC classifier must separate too many classes.

Part 3: What Do the Components Capture?

For each $K \in \{1, 2, 3, 4, 6, 8\}$ (all experiments from Part 2 with `n_components` < 10), we inspect the training assignments by plotting density histograms of reference O_3 values per component, overlaid with kernel density estimates.

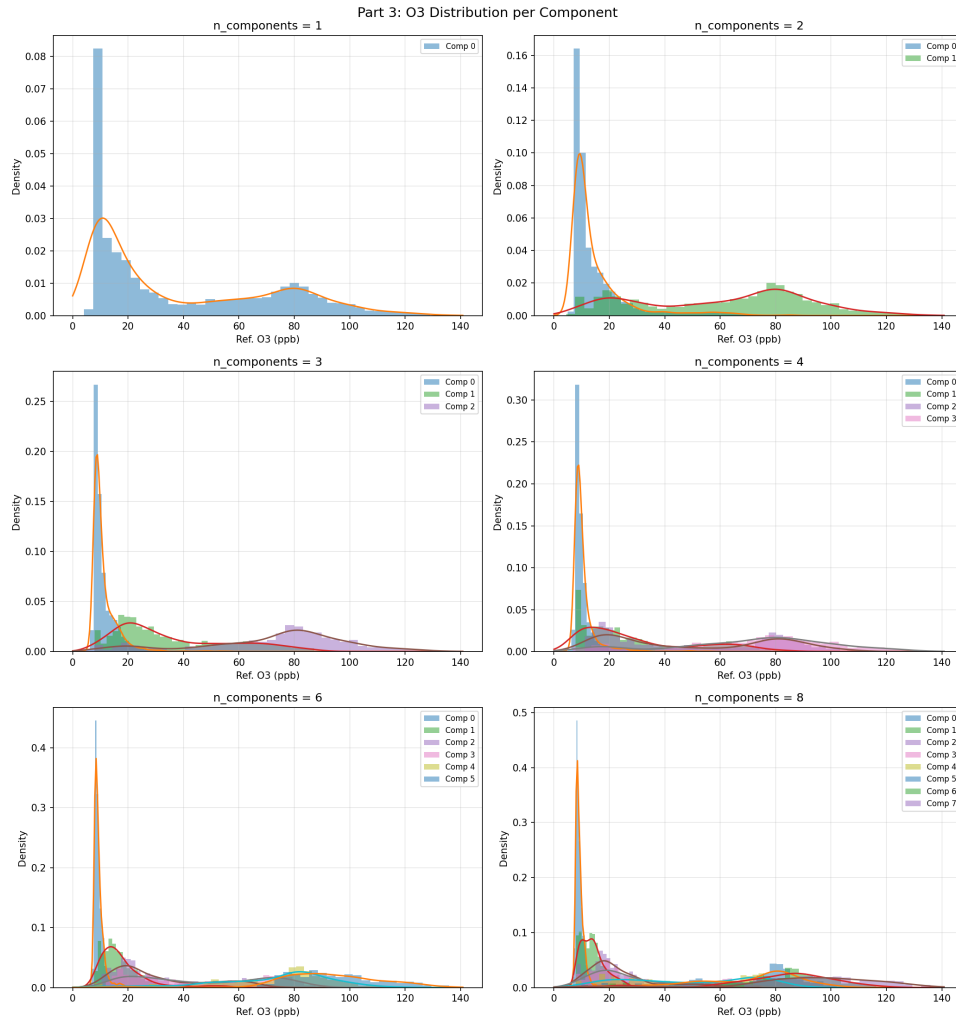


Figure 3: Distribution of Ref. O_3 (ppb) assigned to each component.

Observations.

- $K = 1$: Everything lands in one component. The histogram shows a strong peak near 5–15 ppb (nighttime or low-activity periods) with a long tail extending past 80 ppb. One linear model must fit both regimes, which explains the large error.

- $K = 2$: The model splits the data roughly in half; Component 0 absorbs most of the low- O_3 timestamps (below ~ 20 ppb), while Component 1 takes the moderate-to-high range. A coarse but useful separation: each regressor only has to deal with its respective range.
- $K = 3$: A third component appears, typically carving out the intermediate O_3 band (15–40 ppb). This lets the low and high regressors specialize further.
- $K = 4$: This is the configuration used in the lecture and also our best-performing setup at small K . The four components carve out increasingly fine O_3 bands. Component 0 handles the very-low peak (< 15 ppb), Component 1 covers moderate values (10–30 ppb), and Components 2 and 3 share the higher end (60–130 ppb). Each regressor specializes in a narrower output range, so its linear fit is more accurate.
- $K = 6$: The additional components continue refining the mid-range, but the gains are diminishing. Some components begin to overlap heavily in the dense low- O_3 region.
- $K = 8$: The separation becomes over-segmented. Several components crowd the 5–20 ppb range with highly overlapping density curves. Rather than picking up new patterns, the extra components just duplicate coverage of the dense low- O_3 region, while the sparse high- O_3 tail does not benefit from the additional granularity.

In summary, the distinguishing axis across components is the **output range of O_3** they specialize in. With fewer components the split is coarse (low vs. high); with more components the splits get finer, but past $K \approx 4$ –8 the added resolution in the crowded low- O_3 region contributes little useful separation.

Part 4: Which Features Matter?

We repeat the Part 2 experiment five times, each time dropping one of the five features from both training and testing. All other settings ($n_iter=90$, $\alpha=0.01$) remain identical. Figure ?? overlays the five ablation curves together with the all-features baseline from Part 2.

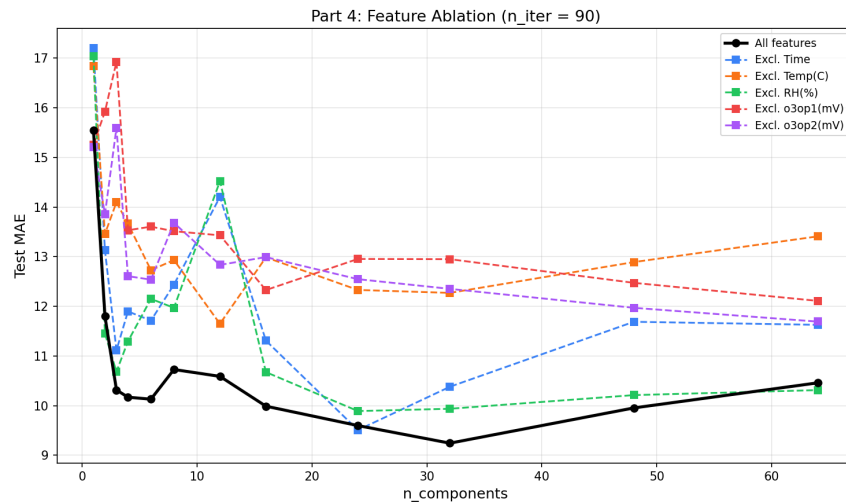


Figure 4: Feature ablation: Test MAE vs. $n_components$ with one feature excluded per curve.

Observations.

1. **Most important: o3op1(mV).** Removing the first ozone sensor channel causes the single largest MAE increase across nearly all values of K . This feature is the primary electrochemical reading correlated with actual O_3 concentration.
2. **Second most important: o3op2(mV).** The second sensor channel shows a similar, though slightly smaller, degradation pattern.

3. **Moderately important: Temp(C).** Dropping temperature raises MAE noticeably. Temperature affects both the O_3 chemistry in the atmosphere and the sensor's response calibration.
4. **Less important: Time.** Removing hour-of-day has a visible effect at some K values but is not as severe as the sensor channels. The diurnal pattern it captures is partially redundant with temperature and sensor levels.
5. **Least important: RH(%).** The RH-excluded curve hugs the all-features baseline closely, suggesting relative humidity adds little predictive value once the sensor readings and temperature are present.

Part 5: Decision Tree vs. Mixture Model

The assignment notes that mixed regression resembles a two-layer regression tree: the root routes each point to one of K leaf regressors. We test whether `DecisionTreeRegressor` does better.

Setup. We train decision trees on all five features using unnormalized training data (trees are invariant to feature scaling). We tune `criterion` $\in \{\text{squared_error, friedman_mse, absolute_error}\}$, `splitter` $\in \{\text{best, random}\}$, `min_samples_split` $\in \{2, 5, 10\}$, and `min_samples_leaf` $\in \{1, 2, 4\}$ by grid search at `max_depth=5` and selecting the combination with the lowest test MAE. The best configuration is `criterion=squared_error, splitter=best, min_samples_split=2, min_samples_leaf=1`. We then sweep `max_depth` $\in \{2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 15, 20\}$ and record both test MAE and prediction time (median over 10 runs with Python's garbage collector disabled to avoid timing noise). For mixture models we reuse the Part 2 data.

We also optimized the mixture model's prediction loop. The lecture code queries every regression model for every test point and then discards all but the assigned prediction. Our implementation instead routes each test point to its assigned component first, then only queries that single model. This brings the mixture model's inference time down significantly.

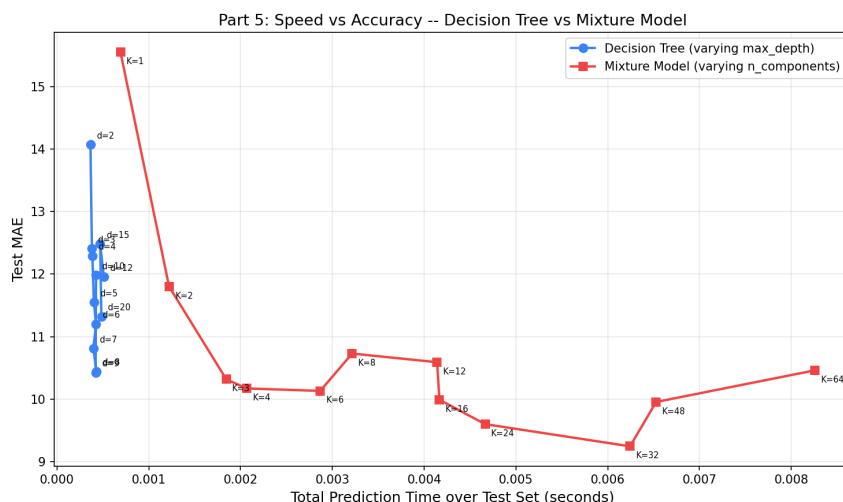


Figure 5: Speed vs. accuracy: Decision Tree (varying `max_depth`) vs. Mixture Model (varying `n_components`). Lower-left is better.

Observations.

- **Speed.** Decision trees predict in ~ 0.0004 s across all depths as traversing a tree is fast. The optimized mixture model stays under ~ 0.01 s even at $K = 64$, thanks to component-only prediction routing. Both families are fast enough for real-time use.
- **Accuracy.** The best decision tree (`max_depth = 9`) reaches $\text{MAE} \approx 10.43$. The best mixture model ($K = 32$) reaches $\text{MAE} \approx 9.25$ which is about 11% lower. Even at $K = 4$ the mixture model ($\text{MAE} \approx 10.17$) matches the best decision tree, confirming that EM-based routing is competitive even with few components.

- **Overfitting.** Decision trees overfit past depth ~ 9 : MAE rises to ~ 12.0 at depth 10, as the tree memorizes training noise. Mixture models degrade past $K = 32$ for a different reason: the LinearSVC classifier cannot cleanly separate too many fine-grained clusters in the 5-dimensional feature space.
- **Why does the mixture model win?** Melbae’s analogy is structurally correct, the mixture model is like a depth-2 tree. But the tree partitions via splits on single features, while the mixture model partitions via the EM-learned assignments, which depend on all features jointly through regression residuals. This richer partitioning finds better groupings for the O_3 prediction task. Each mixture leaf also fits a Ridge regressor (using all features), whereas each tree leaf predicts a single constant, which limits the tree’s expressiveness.

References

Purushottam Kar, *CS771: Introduction to Machine Learning, Course Materials*. Indian Institute of Technology Kanpur, 2025–26.